ANDS Reference Guide

Anomaly-based Network Detection System — what it does, its modules, and its commands

---

What ANDS actually does when it "scans"

ANDS does not scan a website or search for a single vulnerability the way a web scanner does. It watches network traffic — the stream of data packets moving across a network interface (like your computer's connection to the Metasploitable2 VM) — and looks for signs that something unusual is happening.

There are two different things ANDS does that both get called "scanning":

1. Capturing traffic (listening)

This means turning on the network interface and recording every packet that passes through it for a set amount of time. A packet is a small chunk of data — every web request, ping, or file transfer is broken into many packets. ANDS counts how many packets arrive, how big they are, and how fast they're arriving, in small time slices (e.g. every 3 seconds). This builds a "baseline" — a profile of what normal traffic looks like on that network.

2. Detecting anomalies (comparing)

Once ANDS knows what "normal" looks like, it compares new traffic against that baseline in two different ways:

• **Statistical detection (z-score):** ANDS calculates how far a burst of traffic deviates from the normal average, in standard deviations. A huge, sudden spike in packet rate (like a flood attack) stands out mathematically, even if ANDS has never seen that specific attack before.
• **Pattern detection (port scan):** ANDS watches how many different ports on the target one source computer tries to contact. A normal computer talks to one or two ports. A computer probing 100+ different ports in a short time is almost certainly running a port scan — a common first step attackers use to find open, exploitable services.

In short: ANDS listens to traffic, learns what's normal, and flags anything that looks statistically unusual or matches a known attack pattern — not a website vulnerability scan, a network behavior monitor.

Modules currently built

capture/baseline

Turns on the network interface and records traffic for a set duration. Breaks the capture into time windows and calculates packet rate and average packet size per window. Saves this as the "baseline" file that detection modules compare against. Can also save the raw capture to a .pcap file (viewable in Wireshark) or read from an existing .pcap instead of capturing live.

detect/zscore

Reads the baseline file and calculates how far each time window's packet rate deviates from the average, in standard deviations (the "z-score"). Flags any window that crosses a set threshold as an anomaly. Good at catching sudden, sharp spikes like flood attacks; less reliable against slow, spread-out activity.

detect/portscan

Watches live traffic and tracks how many distinct destination ports each source computer contacts. If one source touches more ports than the set threshold, it's flagged as a likely port scan. This is pattern-based, not statistical — it caught the test scan reliably even when z-score did not.

generate/synthetic

Sends fake traffic on demand — normal-looking pings, a simulated port scan, or a simulated flood — toward a target IP. Used to test detection modules without needing real attack traffic, and as a safety net for live demos if the real network is quiet.

report/generate

Builds a PDF report summarizing the session: how many alerts were raised and a table listing each one (source, type, and detail like z-score or port count).

system/selftest

Runs a small set of built-in checks against known values — e.g. does the z-score math correctly flag an obvious spike, does it correctly ignore normal variation. Confirms the detection logic itself is working correctly, independent of any live network.

*custom/**

Any Python file dropped into the modules/custom/ folder is automatically picked up by the reload command and made available to use, without touching any of the core code. Lets the tool be extended with new detection ideas later.

Commands

Command	What it does
use <module>	Loads a module so you can configure and run it. Example: use detect/zscore
show modules	Lists every module currently available, numbered.
show options	Lists the currently loaded module's settings, their current values, and whether each is required.
set <option> <value>	Changes a setting on the currently loaded module. Example: set THRESHOLD 3.0
setg <option> <value>	Changes a global setting that stays set across every module, instead of retyping it each time. Exar
run	Executes the currently loaded module with its current settings.
back	Deselects the current module, returning to the main prompt.
uniq	Summarizes all alerts raised so far in this session, collapsing repeats from the same source into or
reload	Rescans the modules/custom/ folder for newly added custom modules.
exit / quit	Closes ANDS.

Status as of this session: capture, both detection modules, synthetic traffic generation, self-test, custom module loading, JSON alert output, and colored console output are all built and confirmed working live against the Metasploitable2 VM. Remaining polish items: gobuster-style live formatting, loading spinner, save/delete prompt, search command, Linux passthrough, robustness pass, and packaging.